Sprint 4 · Concierto, presentación final y defensa del diseño

Objetivo del sprint	Dar conciertos que gasten energía y canciones, preparar la presentación del grupo y defender las decisiones de diseño del programa.
Etapas que cubre	Final de la Etapa 5 y Reto final
Puntos comprometidos	18 puntos en 5 historias
Incremento esperado	Programa completo `manager.c`, sin advertencias, más la tabla de justificación de cada función.

Sprint Backlog

ID	Historia	Tareas sugeridas	Pts	Responsable
HU-21	Dar un concierto	Definir fórmula de fans y gasto de energía · Escribir `dar_concierto` · Descontar vigencia y avisar al llegar a 0 · Probar varios conciertos seguidos	5	5
HU-22	Explicar la equivalencia entre arreglos y apuntadores	Imprimir con `%p` las direcciones de las expresiones equivalentes · Preparar la explicación	2	2
HU-23	Mostrar la canción de la líder sin fallar	Escribir `mostrar_cancion_lider` con tres verificaciones · Probar los tres escenarios	3	3
HU-24	Preparar la presentación del grupo	Escribir `preparar_presentacion` · Llamarla varias veces y revisar rangos · Probar sin líder y sin canción	5	5
HU-25	Justificar el diseño de cada función	Armar la tabla de funciones · Revisar `const` en cada firma · Compilación final limpia · Ensayar la defensa	3	3
		Total	**18**	18

Sprint Review: se acepta el sprint si…

- Tras varios conciertos, la vigencia de la canción compartida baja y, al llegar a 0, deja de dar bono.
- `preparar_presentacion` llamada varias veces nunca deja energías fuera de 0 a `energia_max`.
- El programa no falla si el grupo no tiene líder o si la líder no tiene canción.
- Cada integrante explica por qué una función recibe apuntador y por qué lleva o no `const`.

Riesgos y consejos

- Olvidar `limitar` en `dar_concierto` y dejar energías negativas.
- Desreferenciar `g->lider` cuando es NULL.
- Dejar la defensa para el último minuto: la tabla de funciones se va construyendo durante todo el proyecto.

HU-21 · Dar un concierto — Prioridad Alta · 5 pts

Como manager del grupo, **quiero** que el grupo dé un concierto, **para** que las integrantes activas ganen fans y las canciones se gasten.

Criterios de aceptación

- `dar_concierto` recorre a las integrantes y solo participan las activas.
- Cada activa gana fans según su `popularidad_total` (el equipo documenta la fórmula, por ejemplo `fans += popularidad_total * 10`) y consume energía con `limitar`.
- Si la idol tiene canción vigente, esa canción pierde una presentación y `vigencia` nunca queda negativa.
- Al llegar la vigencia a 0 se imprime que la canción ya no está vigente.
- Tras varios conciertos, la vigencia de la canción compartida baja y, al llegar a 0, deja de dar bono. El equipo documenta en un comentario cuántas presentaciones pierde por concierto (según el enunciado, una por cada integrante activa que la tiene).
- Si `g` es NULL, la función no falla.

Depende de: HU-15, HU-16 · Sprint 4 · Entregable: `void dar_concierto(Grupo *g)`

HU-22 · Explicar la equivalencia entre arreglos y apuntadores — Prioridad Baja · 2 pts

Como estudiante, **quiero** explicar por qué `g->miembros[i]` equivale a `*(g->miembros + i)`, **para** demostrar que entiendo la aritmética de apuntadores.

Criterios de aceptación

- Una demostración (en `main` o en comentarios) muestra que `&g->miembros[i]` y `g->miembros + i` dan la misma dirección.
- La explicación dice que el nombre de un arreglo equivale a la dirección de su primer elemento.
- La explicación dice que sumar 1 a un apuntador avanza un elemento completo (una `Idol`), no un byte.

Depende de: HU-18 · Sprint 4 · Entregable: Demostración con `%p` + explicación

Reto final · Preparar la presentación

Integración: preparar al grupo para una presentación y generar un resumen de su estado. **Sprint 4.**

HU-23 · Mostrar la canción de la líder sin fallar — Prioridad Alta · 3 pts

Como manager del grupo, **quiero** ver la canción de la líder aunque falte algún dato, **para** que el programa nunca se detenga por un NULL.

Criterios de aceptación

- `mostrar_cancion_lider` imprime `g->lider->cancion->nombre` cuando todo existe.
- Antes de cada flecha se verifica: ¿existe el grupo?, ¿hay líder?, ¿la líder tiene canción?
- Si no hay grupo o líder, imprime un mensaje que lo indica; si la líder no tiene canción, imprime un mensaje con su nombre.
- Se prueba con tres escenarios: líder con canción, líder sin canción y grupo sin líder.

Depende de: HU-17 · Sprint 4 · Entregable: `void mostrar_cancion_lider(const Grupo *g)`

HU-24 · Preparar la presentación del grupo

Prioridad Alta · 5 pts

Como manager del grupo, **quiero** preparar al grupo para una presentación y ver un resumen de su estado, **para** saber si está listo para salir al escenario.

Criterios de aceptación

- `preparar_presentacion` hace descansar a la integrante más cansada (verificando NULL), da el concierto y muestra el resumen.
- El resumen incluye a las integrantes, cuántas siguen activas y la canción de la líder.
- Llamar varias veces a la función nunca deja una energía fuera del rango 0 a `energia_max`.
- Funciona sin errores si el grupo no tiene líder o si la líder no tiene canción.

Depende de: HU-18, HU-19, HU-20, HU-21, HU-23 · Sprint 4 · Entregable: `void preparar_presentacion(Grupo *g)`

HU-25 · Justificar el diseño de cada función

Prioridad Media · 3 pts

Como estudiante, **quiero** explicar para cada función por qué recibe un apuntador y por qué lleva o no `const`, **para** demostrar que entiendo los conceptos de la práctica.

Criterios de aceptación

- Se entrega una tabla con todas las funciones: tipo de parámetro (valor, apuntador o apuntador `const`), si modifica o solo lee, y la razón.
- Las funciones que modifican no llevan `const`; las que solo leen sí, con la excepción justificada de `idol_mas_cansada`, que recibe `Grupo *` porque devuelve un apuntador modificable a un elemento.
- No hay variables globales y el programa final compila con `gcc -Wall -Wextra -o manager manager.c` sin advertencias.
- Cada integrante del equipo defiende al menos una función en la Sprint Review final.

Depende de: HU-24 · Sprint 4 · Entregable: Tabla de funciones + compilación final limpia